

Basic Open-Source Soft Ware, Section 2

Build Lotto Site Document

GitHub: <https://github.com/YooJunHyuk123/Build-Lotto-Site>

Yoo, J. H.

32212808

Department of Mobile Systems Engineering

2025-11-01

Table of Contents

1. 서론
2. 소프트웨어 요구사항 기술 (Requirements)
3. 소프트웨어 설계 (Design)
4. 소프트웨어 구현 (Implementation)
5. 시험 & 디버깅 (Verification)
6. 소프트웨어 유지보수 (Maintenance)
7. 결론
8. 참고 문헌

1. 서론

A. 개발 목적

본 프로젝트의 목적은 Django 웹 프레임워크와 Docker 컨테이너 기술을 활용하여 웹 애플리케이션을 개발하고 배포하는 과정을 학습하는 것이다. 이를 위해 로또 사이트가 개발 대상으로 선정되었다.

2. 소프트웨어 요구사항 기술 (Requirements)

A. 기능적 요구사항

회원 가입 & 로그인 & 로그아웃 기능
회원 가입, 로그인, 로그아웃 기능을 제공해야 한다.
로또 구매 기능
사용자는 번호를 직접 선택해 로또를 구매할 수 있어야 한다. 사용자는 번호를 자동 선택해 로또를 구매할 수 있어야 한다.
당첨 조회 기능
사용자는 당첨 여부, 선택한 번호, 당첨 번호를 조회할 수 있어야 한다.
관리자 로그인 & 로그아웃 기능
관리자로 로그인, 로그아웃 기능을 제공해야 한다.
로또 추천 기능
관리자는 로또를 추천할 수 있어야 한다.
당첨자 조회 기능
관리자는 당첨자, 판매 실적을 조회할 수 있어야 한다.

B. 비기능적 요구사항

에러 처리
입력된 인자가 잘못된 경우, 오류 메시지를 출력해야 한다.
유지보수성
소스 코드는 Django 의 MVT (Model View Template) 구조로 모듈화해야 한다. 환경은 Docker 를 통해 일관성을 유지해야 한다.
보고서
보고서의 표지에서 소스 코드가 있는 GitHub 링크를 기술해야 한다. 보고서의 소프트웨어 구현 부분에서 개발과 테스트 과정과 용어 설명을 상세하게 기술해야 한다. 보고서에 참고 문헌을 기술해야 한다.

3. 소프트웨어 설계 (Design)

A. 아키텍처 설계

본 프로젝트는 소프트웨어 요구사항의 비기능적 요구사항인 '소스 코드는 Django 의 MVT (Model View Template) 구조로 모듈화해야 한다.'는 조건이 있기 아키텍처 설계를 따로 하여 변경하지 않는다.

B. UI (User Interface) & UE (User Experience) 설계



C. 데이터베이스 설계

본 프로젝트에서는 사용자가 구매한 로또 번호와 관리자가 추첨한 로또 번호를 데이터베이스에 저장하여 사용한다.

4. 소프트웨어 구현 (Implementation)

A. 개발 환경 & 도구

Operating System
Microsoft Windows 11 Pro
Kernel
10.0.26200
IDE (Integrated Development Environment) & Editor
Git Bash, Visual Studio Code
Programming language
Python 3.10-slim
Framework
Django 5.2.7
Build tool
Docker, docker-compose
Data Base
SQLite3

B. 모듈별 기능 설명

home.html
홈 페이지다.
signup.html
기능적 요구사항의 회원 가입이 구현된 페이지다.
login.html
기능적 요구사항의 로그인 구현된 페이지다.
logout.html
기능적 요구사항의 로그아웃을 실행한 경우 연결되는 페이지다.
buy.html
기능적 요구사항의 로또 구매가 구현된 페이지다.
check.html
기능적 요구사항의 당첨 조회가 구현된 페이지다.
lotto.html
기능적 요구사항의 로또 추첨이 구현된 페이지다.
admin_check.html
기능적 요구사항의 당첨자 조회가 구현된 페이지다.

C. 주요 알고리즘 설명 & Code Snippet

```
python -m django startproject lotto_project
cd lotto_project
python manage.py startapp lotto_app
```

위 명령어로 Django 프로젝트와 앱을 생성하였다.

```
from django.db import models
from django.contrib.auth.models import User
import random

# 사용자가 구매한 로또 번호
class LottoTicket(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE)
    numbers = models.CharField(max_length=50)
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f"{self.user.username} - {self.numbers}"

# 관리자가 추첨한 로또 번호
class LottoDraw(models.Model):
    numbers = models.CharField(max_length=50)
    draw_date = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.numbers
```

위와 같이 models.py 파일을 수정하였다.

models.py 파일은 데이터베이스의 구조를 정의하는 파일로 ORM (Object Relational Mapping)을 이용해 테이블과 컬럼을 정의한다.

본 프로젝트에서는 사용자가 구매한 로또 번호와 관리자가 추첨한 로또 번호를 데이터베이스에 저장하여 사용한다.

```

# 생략

# 홈 페이지 (사용자와 관리자 모두 사용)
def home(request):
    return render(request, 'lotto_app/home.html')

# 회원가입 페이지 (사용자만 사용)
def signup_view(request):
    # 생략

# 로그인 (사용자와 관리자 모두 사용)
def login_view(request):
    # 생략

    # 관리자 계정으로 로그인한 경우 로또 추첨 페이지로 이동
    # 생략

# 로그아웃 페이지 (사용자와 관리자 모두 사용)
def logout_view(request):
    # 생략

# 로또 구매 페이지 (사용자만 사용)
def buy_lotto(request):
    # 생략

# 당첨 조회 페이지 (사용자만 사용)
def check_lotto(request):
    # 생략

# 로또 추첨 페이지 (관리자만 사용)
def lotto_draw(request):
    # 생략

# 당첨자 조회 페이지 (관리자만 사용)
def admin_check(request):
    # 생략

```

위와 같이 views.py 파일을 수정하였다.

views.py 파일은 URL 로 들어오는 사용자의 요청을 처리하고 어떤 template 파일을 사용할 것인지 지정하고 응답을 생성하는 파일이다.

본 프로젝트에서는 사용자와 관리자가 사용하는 페이지가 다르다는 것을 고려하여 views.py 파일을 작성하였다.

home.html, signup.html, login.html, logout.html, buy.html, check.html, lotto.html, admin_check.html

위와 같이 template 파일들을 생성하였다.

template 파일들에 대한 내용은 소프트웨어 구현의 모듈별 기능 설명과 동일하기 때문에 생략하고 넘어가도록 하겠다.

```
from django.contrib import admin
from django.urls import path, include
```

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('lotto_app.urls')),
]
```

위와 같이 urls.py 파일을 수정하였다.

urls.py 파일은 URL 과 앱의 매핑 연결을 처리하는 파일이다.

예를 들어 path('', include('lotto_app.urls')) 코드가 실행되면 lotto_app/urls.py 로 요청이 전달된다.

```
from django.urls import path
from . import views
```

```
urlpatterns = [
    path('', views.home, name='home'),
    path('signup/', views.signup_view, name='signup'),
    path('login/', views.login_view, name='login'),
    path('logout/', views.logout_view, name='logout'),

    # 사용자 관련 기능
    path('buy/', views.buy_lotto, name='buy_lotto'),
    path('check/', views.check_lotto, name='check_lotto'),

    # 관리자 관련 기능
    path('lotto/', views.lotto_draw, name='lotto_draw'),
    path('admin_check/', views.admin_check,
name='admin_check'),
]
```

위와 같이 lotto_app/urls.py 를 생성하였다.

lotto_app 내부의 urls.py 파일은 앱 내부의 URL 과 view 를 연결하는 파일이다.

예를 들어 /signup/는 singup_view 로 연결된다.

```
from django.contrib import admin
from .models import LottoTicket, LottoDraw
```

```
# 모델을 관리자 페이지에 등록
admin.site.register(LottoTicket)
admin.site.register(LottoDraw)
```

위와 같이 admin.py 파일을 수정하였다.

admin.py 는 모델을 관리자 페이지에 등록하는 파일이다.

모델을 관리자 페이지에 등록함으로써 관리자 페이지에서 로또 구매 기록과 추첨 결과를 관리할 수 있게 된다.


```
# 생략

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'lotto_app',
]

# 생략
```

위와 같이 settings.py 파일을 수정하였다.
settings.py 파일의 INSTALLED_APPS 에 lotto_app 을 추가해야 사용이 가능하다.

```
# Python 이미지 사용
FROM python:3.10-slim

# 작업 디렉터리
WORKDIR /app

# 종속성 복사 및 설치
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# 프로젝트 복사
COPY . .

# 포트 설정
EXPOSE 8000

# 서버 실행
CMD ["python", "manage.py", "runserver", "0.0.0.0:8000"]
```

위와 같이 Dockerfile 파일을 생성하였다.
Docker 파일은 애플리케이션을 실행하기 위한 컨테이너의 이미지를 어떻게 생성할 것인지 설정하는 파일이다.
본 프로젝트에서는 Python 이미지를 기반으로 사용하였다.

```
version: '3.9'
services:
  web:
    build: .
    container_name: lotto_project-web
    ports:
      - "8000:8000"
    volumes:
      - ../app
```

위와 같이 docker-compose.yml 파일을 생성하였다.
docker-compose.yml 파일은 web 서비스로 Django 서버를 실행하고, 포트를 매핑하고, 볼륨을 공유하도록 설정하는 파일이다.
이를 통해 여러 개의 컨테이너를 한 번에 관리할 수 있다.

Django==5.2.7
위와 같이 requirements.txt 파일을 생성하였다. requirement.txt 파일은 프로젝트 실행에 필요한 Python package 를 명시한 파일이다. SQLite 의 경우에는 Python 의 표준 라이브러리에 포함되기 때문에 별도로 명시하지 않았다.
docker-compose run web python manage.py makemigrations
위 명령으로 마이그레이션을 생성하고 적용하였다.
docker-compose run web python manage.py createsuperuser
위 명령어로 관리자 계정을 생성하였다.
docker-compose up
위 명령으로 Docker 로 Django 서버를 실행하였다.

5. 시험 & 디버깅 (Verification)

시험 목록	입력	출력	상태
홈 페이지 접근	http://localhost:8000/	홈 페이지로 이동	Pass
회원 가입	새 사용자 정보 입력	홈 페이지로 이동	Pass
로그인	기존 사용자 정보 입력	로또 구매 페이지로 이동	Pass
로그아웃	로그아웃 버튼 클릭	로그아웃 페이지로 이동	Pass
로또 구매	수동 또는 자동 선택	당첨 조회 페이지로 이동	Pass
관리자로 로그인	관리자 정보 입력	로또 추첨 페이지로 이동	Pass
로또 추첨	추첨 버튼 클릭	당첨자 확인 페이지로 이동	Pass

6. 소프트웨어 유지보수 (Maintenance)

모듈화 설계 & 확장성
기능을 각각의 모듈로 분리하였다. 단일 책임 원칙 (SRP, Single Responsibility Principle)을 준수하여 코드 수정 시 다른 모듈에 대한 영향을 최소화했다.
에러 처리
잘못된 인자가 입력된 경우 적절한 에러 처리를 수행하도록 하였다.
코드 가독성
모든 모듈과 함수에 대한 주석을 작성하여 코드 이해도를 높였다.

7. 결론

Django 웹 프레임워크와 Docker 컨테이너 기술을 활용하여 웹 애플리케이션을 개발하고 배포하는 과정을 알게 되었다.

클라우드 컴퓨팅 서비스인 AWS 를 사용해본 경험이 있는데, Docker 는 운영체제의 커널을 공유하는 클라우드 컴퓨터처럼 느껴졌다. 반면 사용 경험이 없는 Django 는 어려웠다.

프론트 엔드의 UI 와 예외 처리가 부족한 점이자 개선이 필요한 점이라고 생각한다.

8. 참고 문헌

Django 공식 문서: <https://www.djangoproject.com/>

Docker 공식 문서: <https://docs.docker.com/>

SQLite 공식 문서: <https://www.sqlite.org/index.html>